

Take-Home: Own the Recommendation Model

An ML-Engineering challenge — fine-tuning a small model to recommend Wohngebäude insurance

"A model can only practice a workflow it can actually perform, fail at, and retry."

Afori is an AI insurance broker for the German market. One of the most valuable — and most repeated — decisions in the business is deceptively simple: **given a building and the tariffs a comparison engine returns, which product should we recommend?** A broker does this hundreds of times a week. It is high-volume, judgement-heavy, and every recommendation is scored by an outcome we can measure.

That is exactly the shape of task where the winning move is **not** a bigger frontier prompt. It is to **own the model**: take a small open-source model and fine-tune it with reinforcement learning against a scored copy of your own workflow — a *digital twin* — until it matches or beats frontier models on the task, at a fraction of the cost and with none of the vendor lock-in.

Your mission: build the best fine-tuned model you can for **Wohngebäude (WG, residential building) product recommendation**, trained inside the digital twin we give you, and prove it against a held-out, expert-annotated benchmark.

The digital twin

We hand you a working copy of the real workflow — the same engine, the same reward, the same stakes:

Piece	What it is	File
Quote tool	The live Mr-Money "Rechenkern" pricing engine, WG only. A risk in → real tariffs out.	<code>tools/calculate_quotes_wg.py</code>
Risk-context tool	Turns a risk into natural-hazard exposure + coverage-emphasis hints. Context to <i>reason</i> with, not the answer.	<code>tools/assess_wg_risk_context.py</code>
Reward / scorer	The value-for-money reward every recommendation is scored by. This is your RL signal.	<code>tools/reward.py</code>
Golden benchmark	50 real WG scenarios, priced live, with the top-3 products ranked by Fable (a frontier model) as the known-correct answers. Held out — evaluation only.	<code>data/golden_wg_recommendations.jsonl</code>

You act inside the twin the way a broker would: pull the risk context, price the building, weigh coverage against price, and commit a recommendation. The twin scores it. The gap between your model's choice and the expert answer is the signal your fine-tune has to close.

See `PROBLEM_FRAMING.md` for the full methodology, and `tools/README.md` for how to run each tool.

What you build

A pipeline that produces a **fine-tuned small open-source model** which, given a WG risk and its priced tariffs, recommends the best product(s).

You choose the recipe. A strong submission will typically include:

1. **Rollout generation** — use `calculate_quotes_wg` + `assess_wg_risk_context` to build training episodes (you generate your own training scenarios — the 50 golden ones are for evaluation only).
2. **A reward-driven fine-tune** — RL / GRPO (or a preference/ranking method you justify) on a small open-source base model, optimising the provided reward.
3. **Evaluation** against the held-out golden set: how well does your model's recommendation agree with the Fable-ranked top-3, and how does its reward compare to (a) the reward-argmax baseline and (b) a frontier model prompted zero-shot on the same task?
4. **A cost/quality argument** — tokens/€ per 1,000 recommendations for your model vs. the frontier baseline. Owning the model only wins if it's cheaper *and* competitive.

Baselines to beat

We ran three models zero-shot against the golden set so you have reference numbers (details in `BENCHMARK.md`, raw data in `results/`):

Model	top-1 vs Fable	reward-ratio	€ / 1,000 recs
reward-argmax (the scorer itself)	0.38	1.00	0.00
Qwen3.5-9B (open-source, untuned)	0.42	0.98	€0.19
Gemini 3 Flash (frontier)	0.80	0.76	€0.89
gpt-5.4 (frontier)	0.84	0.80	€1.34

The untuned open-source model captures value-for-money (reward-ratio ≈ 1.0 — it essentially picks the reward optimum) but misses the expert's quality-first judgement (lowest agreement). Frontier models match the expert $\sim 2\times$ as often, at $5\text{--}7\times$ the cost. **Your target: fine-tune the small open-source model from ~ 0.42 up toward frontier-level agreement (~ 0.80) while keeping its $\sim \text{€}0.19$ / 1k cost.** Beating frontier *quality* at open-source cost is the bar.

Deliverables

- **Code:** the training + evaluation pipeline, reproducible from these tools.
- **The fine-tuned model** (weights or adapter) or a clear path to reproduce it.
- **A short write-up (≤ 4 pages):** your approach, the reward you optimised (and any changes you made to it, justified), results on the golden benchmark vs. the two baselines, the cost/quality trade-off, and what you'd do next with more time.
- **An evaluation script** we can run to reproduce your headline numbers against `data/golden_wg_recommendations.jsonl`.

How we evaluate you

Dimension	What we look for
Methodology	A real digital-twin loop: rollouts, a reward you understand, honest evaluation. Not a prompt in a trench coat.
Result	Agreement with the Fable top-3 + reward vs. baselines. Beating frontier at lower cost is the bar.
Reward literacy	Do you understand what the reward rewards — and where it's blind? (Hint: inspect what <code>num_coverages</code> actually varies across a scenario.)
Engineering	Reproducible, readable, sane data handling.
Judgement	Sensible scope, clear trade-offs, honest about limitations.

Rules & notes

- **Time-box: ~1 week, part-time.** We care about the reasoning and the loop, not polish. A smaller model that clearly beats a baseline beats a heroic setup that doesn't.
- **The golden set is held out.** Don't train on `data/golden_wg_recommendations.jsonl`.
- **Credentials for the quote engine** are provided separately (env `MM_ID` / `MM_PA`). Be gentle with the live engine — cache, and space out calls (~3s).
- You may **change the reward** if you can argue it recommends better products — just make the change explicit and evaluate both.
- Use any open-source base model and any RL/fine-tuning stack you like.

Questions are welcome — a good clarifying question is a positive signal, not a negative one.

Problem Framing — Owning the WG Recommendation Model

This document explains the *why* behind the take-home: the business problem, the digital-twin methodology, the reward, and what a strong solution looks like. The candidate-facing brief is in `README.md` ; the tools are in `tools/` .

1. The business problem

Afori is an AI insurance broker for the German market. In the **Wohngebäude (residential building)** line, a lead arrives (a building: postcode, size, age, construction, roof, usage...), a comparison engine (Mr-Money's *Rechenkern*) returns a set of tariffs, and someone has to **recommend** the right product. This decision:

- is **high-volume** — it repeats on every WG lead,
- is **judgement-heavy** — cheapest is rarely best; the right answer trades coverage, deductible, insured sum and price against the building's actual risk,
- and is **measurable** — we can score a recommendation against an expert's, so it is a supervised/RL target, not a vibe.

That combination — repetitive, cognitive, scorable — is precisely where the literature and practice converge on one deployment pattern: **fine-tune a small open-source model with reinforcement learning against a scored copy of the workflow**. It turns knowledge only Afori has (its tools, its data, its notion of a good recommendation) into a model no vendor API can match, at a fraction of the per-token cost of prompting a frontier model on every lead.

2. The digital twin

"We rebuilt the workflow as a digital twin: a simulated environment with the same tools, the same data, and the same stakes as the real thing... so every episode has a known correct answer to score against."

We do the same for WG recommendation. The twin has three parts the model can act in and be scored by:

1. **The real pricing engine** (`calculate_quotes_wg`). Not a mock — the live Rechenkern. A WG risk goes in; real tariffs come out, each with insurer, product, price, insured amount, deductible, coverage flags and the engine's own quality score.
2. **A reasoning tool** (`assess_wg_risk_context`). It converts a risk into natural-hazard exposure (flood / storm-hail / surge) and a coverage-emphasis hint. This is the analyst's context — *which*

covers matter for *this* building — and it is what lifts the task above a trivial numeric argmax. It is a transparent heuristic the candidate can and should improve.

3. **A scorer** (`reward.py`). The value-for-money reward every recommendation is graded by — the RL signal.

An **episode** is one scenario: pull the context, price the building, choose a recommendation, get scored. The model practices thousands of these; the fine-tune learns the policy that maximises the reward and agrees with the expert.

3. The reward — value-for-money

A good WG recommendation is **protection per euro**. The reward encodes exactly that. Within a scenario's quote set, each field is min-max normalised (so scenarios with very different premiums are comparable):

```
protection = w_cov·coverages_n + w_ins·insured_n + w_ded·(1 - deductible_n)
reward      = protection / (1 + w_price · price_n)
```

- more **coverages** and higher **insured amount** raise protection,
- a lower **deductible** raises protection (less customer out-of-pocket),
- a higher **price** penalises the quote (the denominator),
- the recommended quote is `argmax(reward)` .

Defaults: `w_cov 0.4`, `w_ins 0.3`, `w_ded 0.3`, `w_price 1.0` . All configurable.

How coverage enters (and a subtlety to notice). The optional covers are real: every building requests fire + glass, and a building in a flood-exposed postcode also requests natural-hazard (Elementar) — so `num_coverages` (1 or 2) varies across the set **with the building's hazard**, and Elementar-bearing tariffs cost visibly more. But the engine returns only tariffs that satisfy the request, so **within a single scenario coverage is usually uniform** — meaning the *within-scenario* differentiators the reward ranks on are price, deductible, and `coverage_score` (the engine's 0–100 quality score, exposed via the optional `w_score` term, off by default). A strong candidate notices that the intra-scenario coverage term is flat and either weights `coverage_score` or argues for a better coverage term, and evaluates the change. Reward literacy is part of the test.

Where context enters. The base reward is risk-blind. The `assess_wg_risk_context` tool both *drives which covers each building requests* (flood → Elementar) and lets a policy reason *beyond* the reward — e.g. preferring a tariff whose coverage matches the building's hazard. Candidates may fold context into a risk-aware reward; if they do, they should evaluate both.

4. The golden benchmark

50 real WG scenarios, priced live through the twin, each annotated by **Fable** (a frontier model) which independently ranked the **top-3 recommended products with a broker rationale, blind to the numeric reward**. These are the *known correct answers*.

- **Held out.** Candidates evaluate against it; they do not train on it.
- **Two references per scenario:** the reward-argmax (`reward_recommended_tariff_id`) and Fable's expert top-3 (`fable_annotation.top_3`). Where they disagree is the interesting part — a learned policy that captures Fable's judgement *and* scores well on reward is the goal.

Schema of each JSONL line: `id` , `risk` , `risk_context` , `quotes` (each with `reward` + `reward_components`), `reward_recommended_tariff_id` , and `fable_annotation.top_3` (`rank` , `tariff_id` , `insurer` , `product` , `rationale`).

5. What "good" looks like

- A model that **agrees with Fable's top-3** more often than a zero-shot frontier prompt, and whose recommendations **score at least as well on reward** as the argmax baseline.
- **Cheaper:** a defensible tokens/€ per 1,000 recommendations well below a frontier baseline — the whole point of owning the model.
- **Honest evaluation:** baselines included (reward-argmax, frontier zero-shot), limitations named, the reward's blind spots acknowledged.

6. Extension ideas (not required)

- A **risk-aware reward** that uses `coverage_emphasis` to value the covers that matter for the building.
 - Replace the heuristic hazard context with real **ZÜRS / GDV** data.
 - **Multi-objective / ranking** training (recommend a ranked top-3, not one).
 - **Calibration:** have the model express confidence and abstain on close calls.
-

The bar is not a perfect model. It is a real digital-twin loop, an honest number against the golden set, and a small model that beats a frontier baseline at a fraction of the cost.